FUNCTIONAL SPECIFICATION DOCUMENT (FSD)

Metro Ticket Generating System in ServiceNow

Project Name	Metro Ticket Generating System in ServiceNow
Prepared By	Devi k
Platform	ServiceNow
Date	29 September 2026

1. Introduction

Project Title: Metro Ticket Generating System in ServiceNow

Project Owner / Contributor: Devi K

This Functional Specification Document adapts the supplied full-stack documentation format to the actual ServiceNow implementation. Where the supplied template refers to React, Node.js, Express.js, MongoDB, npm commands or REST endpoints, the equivalent ServiceNow component is documented instead; unsupported technologies are not claimed.

2. Project Overview

Purpose

Digitize metro ticket booking through a centralized ServiceNow workflow that captures journey details, calculates fares, stores booking data, generates a digital QR ticket and supports operational verification.

Key Features

- Service Catalog-based ticket booking.
- Source, Destination, Passenger Type and Number of Tickets variables.
- Automated fare/station mapping logic.
- Flow Designer-based request processing.
- Request/RITM tracking.
- Custom Metro Database table for structured booking records.
- QR-based digital ticket presentation.
- Service Portal widget/spModal implementation.
- Roles and ACL-based access control.
- Flow Execution Details for troubleshooting.

3. Architecture

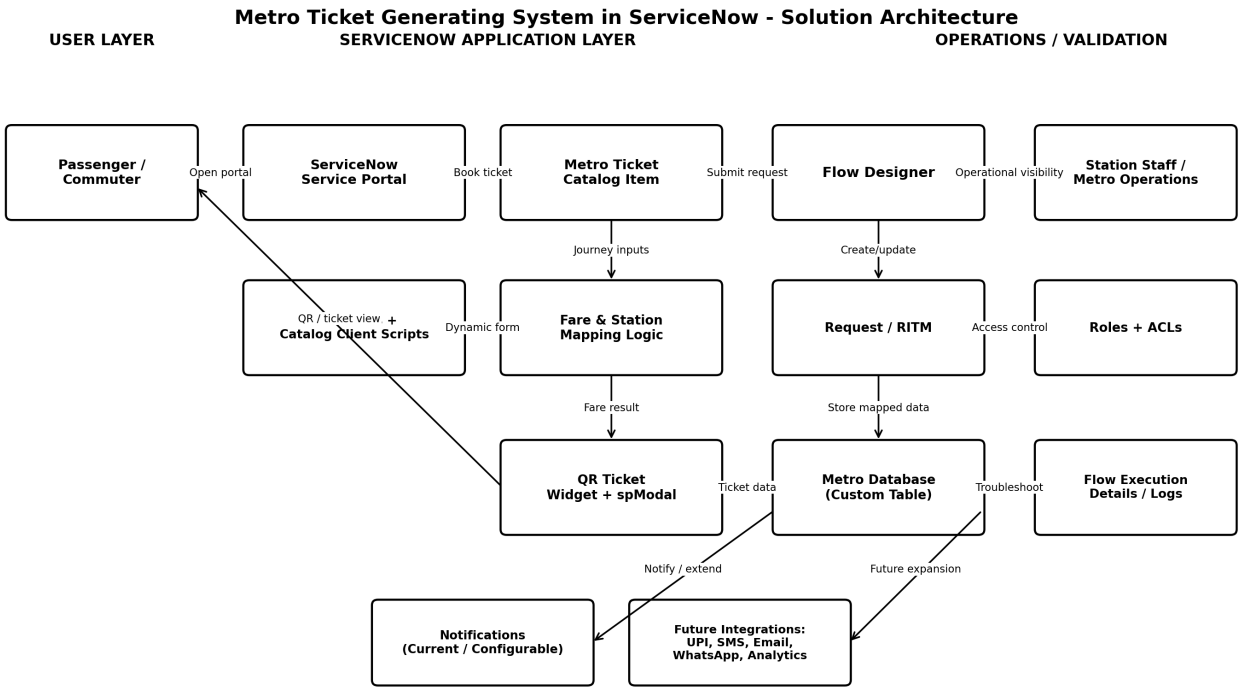


Figure 1. ServiceNow solution architecture

Frontend / User Interface

Equivalent to the frontend layer in the supplied format, the project uses ServiceNow Service Portal and Service Catalog. The passenger interacts with catalog variables, UI Policies and essential Catalog Client Scripts. QR ticket presentation is handled using a Service Portal-compatible widget and spModal.

Backend / Application Logic

The backend equivalent is ServiceNow platform logic. Flow Designer is the primary automation engine. Fare and station mapping logic processes journey inputs, while Request/RITM records maintain the request lifecycle.

Database

ServiceNow records are used instead of MongoDB. The custom Metro Database table stores structured booking/ticket fields, while Request and Requested Item records preserve the service-request context.

4. Setup Instructions

Prerequisites

- Access to a ServiceNow Personal Developer Instance (PDI) or authorized ServiceNow instance.
- Permission to configure Service Catalog, Flow Designer, tables, roles and ACLs.
- Service Portal/widget configuration access where required.
- Test users/roles for requester and operational validation.

Configuration Sequence

1. Create the Metro Ticket Service Catalog item.

2. Create and configure booking variables: Source, Destination, Passenger Type, Number of Tickets, and related fields.

3. Configure station/fare mapping logic.

4. Create the custom Metro Database table and required fields.

5. Build the Flow Designer workflow and variable-to-field mappings.
6. Configure Request/RITM processing and required notifications/tasks.
7. Configure UI Policies and only the essential Catalog Client Scripts.
8. Create/configure the QR ticket widget and spModal presentation flow.
9. Configure roles and ACLs.
10. Run portal, workflow, mapping, QR, security and UAT tests.

5. Configuration / Component Structure

The supplied template contains a source-code folder structure for MERN. This ServiceNow project is configuration-driven, so the equivalent structure is shown below.

Configuration Area	Contents
Service Catalog	Metro ticket catalog item and catalog variables.
Client Configuration	UI Policies and required Catalog Client Scripts.
Automation	Flow Designer flow, actions, conditions and task creation.
Data Model	Request/RITM and custom Metro Database table.
Portal Components	Service Portal page/widget and spModal QR ticket display.
Security	Roles and ACL records.
Operations / Diagnostics	Flow Designer Execution Details, logs and testing records.

6. Running the Application

There are no npm or local server commands because the application runs inside ServiceNow. To use the solution, open the configured ServiceNow Service Portal, navigate to the metro booking catalog item, enter the journey information, submit the request, and follow the generated request/ticket flow.

- Requester path: Service Portal -> Metro Ticket Catalog Item -> enter details -> submit -> track request -> open QR ticket.
- Administrator path: ServiceNow backend -> Flow Designer / tables / security configuration -> monitor executions and records.
- Station/operations path: authorized ticket/record view -> verify booking/QR information.

7. API Documentation

No external REST API endpoint was documented as part of the current implemented scope. The solution primarily uses native ServiceNow platform components and record interactions. Future integrations may expose or consume APIs for payment gateways, SMS, email, WhatsApp services or metro operational systems.

Integration	Current Status	Purpose	Future Interface
Payment Gateway / UPI	Future	Collect digital payment during booking.	External payment API
SMS	Future	Deliver ticket/confirmation.	SMS provider API
Email	Future / configurable notification	Send booking/ticket information.	ServiceNow notification/email service
WhatsApp	Future	Alternative booking/ticket channel.	WhatsApp Business API

8. Authentication & Authorization

Authentication is provided by the ServiceNow instance. Authorization is implemented through roles and Access Control Lists (ACLs). The requester accesses their booking/request experience, while authorized station/operations/admin users access operational records according to configured permissions.

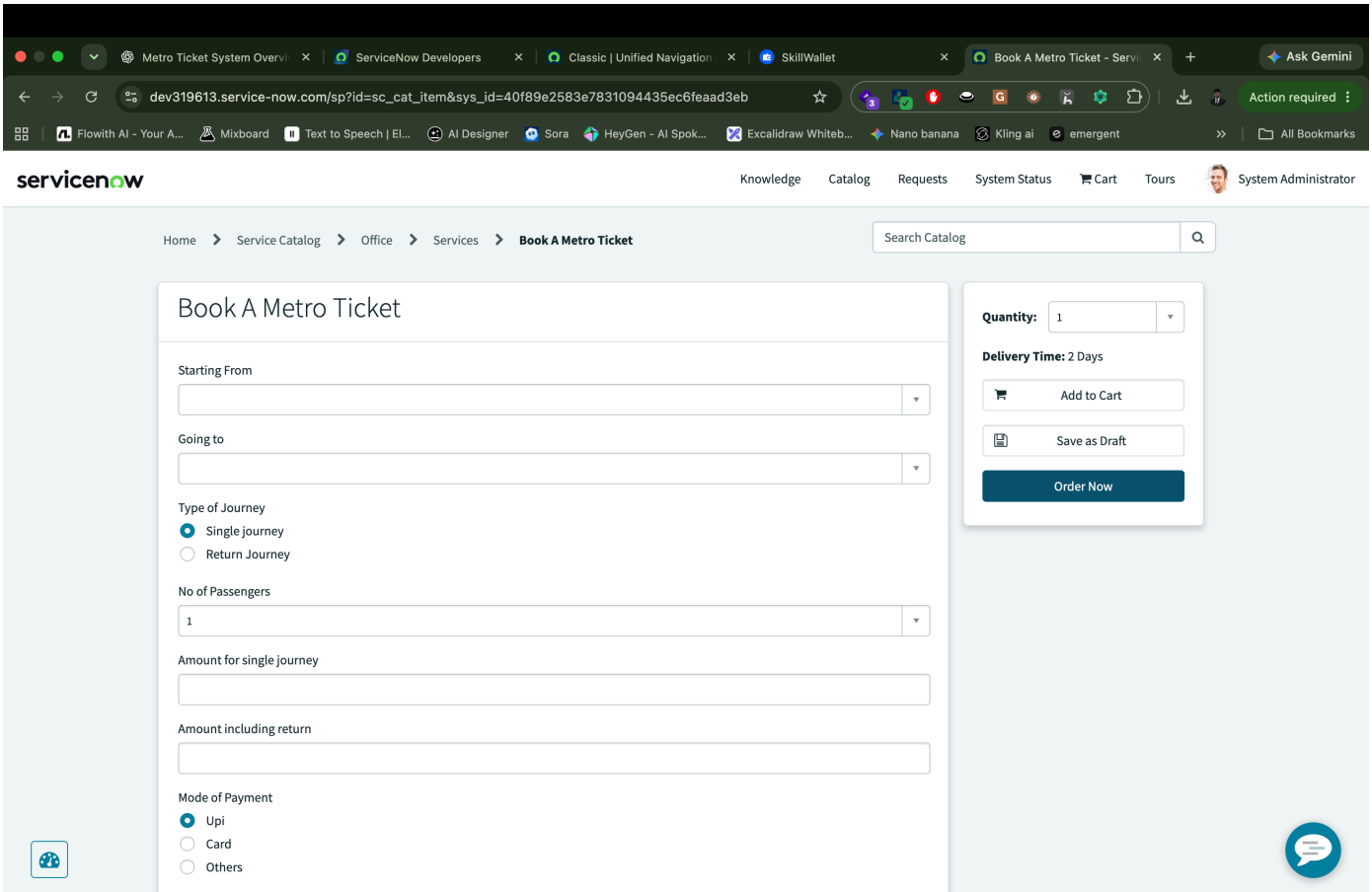
- Passenger/requester: submit ticket request and view permitted request/ticket information.
- Station/operations user: access ticket records required for operational verification.
- Administrator: configure catalog, flow, data model, portal components, roles and ACLs.
- ACL rules: restrict read/write/create/delete access based on user role and record context.

9. User Interface

The primary user interface is the ServiceNow Service Portal catalog experience. The form captures journey data, applies dynamic field behaviour, submits the request and provides ticket/request visibility.

Figure 2. Project workspace / documentation context

Additional implementation screenshots can be inserted for the catalog form, Flow Designer, custom table, QR ticket view, ACL configuration and execution logs when available.



10. Testing

Test Area	Coverage	Status
Catalog form	Source/destination, passenger type, ticket quantity and required fields	Pass
Fare logic	Representative routes and quantities	Pass after mismatch correction

Flow Designer	Request trigger, conditions, record creation and execution path	Pass
Data mapping	Catalog variables to Metro Database fields	Pass after mapping correction
QR ticket	Portal rendering and verification	Pass after widget/spModal fix
Security	Roles and ACLs	Pass
UAT	End-to-end passenger journey	Pass

11. Screenshots or Demo

A final demonstration should show: Service Portal -> Book A Metro Service -> enter journey details -> submit request -> track request -> view stored Metro Database record -> open QR ticket -> validate ticket from the authorized operational perspective.

Project demo link: Not supplied in the documentation set. Insert the final demo video/deployed link before submission if required.

12. Known Issues

Issue / Limitation	Status	Resolution / Note
QR rendering / Service Portal sandbox	Resolved	Implemented widget + spModal architecture.
Fare mismatch	Resolved	Refined fare/station mapping and retested.
Variable-to-field mapping	Resolved	Corrected bindings and population logic.
Flow execution path failures	Resolved	Used Flow Execution Details for diagnosis.
Payment gateway	Not implemented	Future scope.
SMS/WhatsApp ticketing	Not implemented	Future scope.
Production load benchmark	Not measured	No exact production-scale performance benchmark was supplied.

13. Future Enhancements

- Integrated UPI/card/wallet payment.
- Email, SMS and WhatsApp ticket delivery.
- Travel analytics and route-demand dashboards.
- Monthly/season passes, cancellations and refunds.
- Real-time station and service information.
- Offline-friendly ticket access.
- Integration with production metro AFC/validation systems.
- Expanded mobile experience and accessibility improvements.

Appendix - Functional Data Mapping

Data Element	Source	Destination	Purpose
Source Station	Catalog variable	Metro Database source field	Used by fare/station logic.
Destination Station	Catalog variable	Metro Database destination field	Used by fare/station logic.
Passenger Type	Catalog variable	Passenger-type field	Used for booking classification/ fare rules where applicable.

Number of Tickets	Catalog variable	Ticket quantity field	Used in total fare processing.
Fare	Calculated/processed value	Fare field	Stored for ticket and validation.
Request Reference	REQ/RITM	Request reference field	Supports status tracking.
QR/Ticket Data	Generated ticket data	Ticket/QR field or rendered context	Displayed in portal widget/ spModal.